

导论

qwaszx

2026 年 4 月 7 日

今天的内容

- 为什么学 OI / 学 OI 有什么好处?
- 怎么学 OI?
- 回顾一些语法和算法基础
- 对本系列课程的介绍

什么是 OI?

什么是 OI?

面向中学生的**算法竞赛**：设计算法解决问题

通常来说，需要使用**代码**实现算法，最后使用若干组数据进行测试。

什么是 OI?

面向中学生的**算法竞赛**：设计算法解决问题

通常来说，需要使用**代码**实现算法，最后使用若干组数据进行测试。

不是

- 写代码竞赛
 - 不过也可以是（工程题/大模拟）
- 数学竞赛
 - 不需要证明
 - 只需要通过测试

为什么学了 OI

你为什么学了 OI?

为什么学了 OI

你为什么学了 OI?
我为什么学了 OI。

学 OI 的好处

- 提升逻辑思维能力
- 算法思维
 - 如何描述解决问题的步骤
 - 对计算的理解
- 计算机和离散数学知识
- OI 社群
- 更多地玩电脑
- 升学与就业

升学方面：

- 金牌可以保送清华姚班/北大图灵班
- 银牌可以破格入围强基计划
- 省一对强基计划和各种自主招生有一定帮助

就业方面：

- OI/XCPC 竞赛获奖对于计算机相关行业有一定帮助
- 可以继续做教培，相对普通文化课报酬更高
- 对口 TCS（理论计算机科学）

好处说完了，那么坏处呢

- 机会成本：选择 OI 需要放弃一些其他东西
- 更多地使用计算机和互联网，可能会损害身心健康。
- 对于升学和就业的性价比不算很高
 - 目前，银牌相比省一的优势不大，但是困难得多
 - 集训队每年只有 50 人，竞争激烈

总结

初中阶段，适合学文化课有余力并且学 OI 学得开心的同学学习。
高中阶段，功利角度建议根据自身成绩谨慎选择（高一进省队 -> 高二进集训队）
兴趣是第一位的！

OI 比赛的级别

- ① CSP-J
- ② CSP-S
- ③ NOIP
- ④ 省选
- ⑤ NOI
- ⑥ CTS
- ⑦ IOI

其中 CSP-J CSP-S 属于前期，CSP-S NOIP 属于中期，省选 NOI 属于后期，再往后是大后期

做题的过程

- ① 读题
- ② 分析性质
- ③ 抽象问题
- ④ 使用/改编/设计算法来处理
- ⑤ 实现

前期最重要的是 1 和 5，中期最重要的是 3 和 4，后期和大后期最重要的是 2 和 4
学习的过程中，每个部分都在逐步提高

抽象/封装：忽略某个东西的内部结构，只关注其输入和输出
在做题中练习
多看看其他人的写法

抽象问题，调用算法

我们会学习很多的算法。学习的过程中，

- 关注算法整体的功能，熟知一个问题可以被什么算法解决，一个算法可以解决什么问题，有哪些变种和扩展
- 关注算法的每一步，熟知其原理、作用，多思考可以如何扩展
- 关注证明

在做题中练习

多总结

分析性质，设计算法

也就是所谓的人类智慧。一般来说，很难！
多总结题目和算法的模型。相似的模型往往具有相似的性质。
多思考题解/算法的每一步是如何想到的。
尝试建立不同概念和不同算法之间的联系，抽象其共性

训练与学习资源

OJ/题库：洛谷、QOJ、LOJ、UOJ；Codeforces、Atcoder

比赛：Codeforces、Atcoder

学习资源：oiwiki、各种优质博客（如 liu_cheng_ao 推荐的、command_block、Elegia）
各种 AI

OI 社群与搜索引擎。不懂就问/搜！

- 学习知识点
- 刷知识点对应的题
- 对知识点进行归纳总结
- 通过模拟赛进行不同知识点的综合练习/复习

刷题的方法

先自行思考，思考到自己一直没有任何想法为止，然后看题解。这个时间长短是不定的
看题解的过程中也要进行思考：作者说的有没有道理，作者是怎么想到的，我能不能自行
补全剩下的部分，哪些部分还可以优化

想出做法/看懂题解后进行实现，这个过程中不要参考任何现成代码

做完题后看看题解区有没有不同做法，看看其他人的代码都是怎么写的，想想自己的代码
哪些地方能更快/更好写

看看题目是否是**经典问题**，是则需要在归纳总结时**记录**

有一些问题是经典的：在拆掉包装后，其技巧/模型非常泛用。每做完一道题都应该思考这道题是不是这种问题。换句话说，做题后要努力将题目的模型/技巧总结出来。

刷什么题

学习资源中提到的题、自行根据知识点搜索相关题目（如洛谷题单）
重要的是刷题前中后的**思考**而不是刷题本身。所以，非常同质化的题不用多刷，几道就够了（再多的可以口胡）
刷题可以形成题目 $\leftrightarrow$ 模型 $\leftrightarrow$ 算法的条件反射，或者说直觉。

循序渐进，避免一直学太简单/太难的东西

青少年的智力、理解能力会随着年龄提高

熟练掌握 OI 大部分知识点通常需要且只需要几千个小时，相当于 12 年 * 8h（高中脱产/初中或者小学就开始学）

对初中生来说，我非常建议学好文化课，义务教育是必要的。不过可以多和家长老师沟通，放弃重复性过强的文化课练习。OI 以兴趣为主，无需强求。

OI 涉及大量的思考，所以充足睡眠非常重要，建议想办法得到充足睡眠。最起码的原则是不能一天比一天困。

多锻炼。一方面大学也有各种体育测试（如 3km），另一方面人生是场马拉松，健康地活着才是成功

人际交流，线上线下均可

可以用代码实现想法
知道一个东西能不能写、大概是怎么写的、好不好写
不出 bug 地尽量快地写出来
学习更优秀的写法（更好写/更快）

先想再写 vs 边写边想

我的习惯：在写某个部分时，对这个部分的整体结构必须提前想好，细节可以边写边想
抽象/封装的代码设计理念

目前你需要非常熟练的东西

- 顺序、分支、循环结构的程序设计
- 各种变量类型、结构体、数组
- 函数
- 简单的模拟、枚举

如果不熟练，可以先多做做橙题

什么是算法

解决问题的**机械**流程。任何人/机器只要照着做就能成功。
流程如何描述取决于执行者：C++/python 等各种编程语言；人类；图灵机

流程需要的**基本指令**数量，其中每个基本指令花费 1 个单位时间。
什么是**基本指令**？

- word RAM——通常的选择：算术运算、位运算、内存访问
- (不严谨) 人类的一个动作
- 图灵机移动一格（想象数组不能直接访问，而是只能访问上一个或者下一个位置）

现代计算机中，不同指令的速度很不相同，这对算法的实际运行效率有很大影响。可以询问 ai “现代计算机的各种操作的速度如何排序”。目前 CCF 的机器 1s 大概可以跑 10^9 个时钟周期。

大 O 记号

$g(n)$ 是 $O(f(n))$ 意味着：存在 N 和 $C > 0$ ，使得对任何 $n > N$ ，都有 $g(n) \leq C \cdot f(n)$ 。换句话说，当 n 非常大时， $g(n)$ 不会超过常数倍的 $f(n)$ 。

讨论算法复杂度时，我们通常使用渐进记号来描述，其中 n 代表问题规模。直观来说，我们只考虑 $f(n)$ 中关于 n 增长最快的一项，且忽略其常数。

OI 中常见的复杂度： $O(1)$ 、 $O(\log n)$ 、 $O(\sqrt{n})$ 、 $O(n)$ 、 $O(n \log n)$ 、 $O(n \log^c n)$ 、 $O(n^c)$ 、 $O(2^n n^c)$ 、 $O(c^n n^k)$

渐进记号的各种用法

$2^{O(\log n)}$ 是什么意思?

人们在表达式的某个位置中写 $O(f(n))$ 时, 表示: 存在某个常数 $C > 0$, 使得这里的量在 n 非常大的时候不超过 $C \cdot f(n)$ 。

带常数的复杂度: $2 \log n + O(1)$

数学中的例子: $\frac{n}{n+1} = 1 + O(1/n)$ 。(这里是对绝对值用 O)

其他渐进记号

$\Omega(f(n))$: 把大 O 的 $\leq C \cdot f(n)$ 换成 $\geq C \cdot f(n)$

$o(f(n))$: 当 n 趋于无穷时, 与 $f(n)$ 的比值趋于 0, 也就是严格比常数倍 $f(n)$ 小

$\omega(f(n))$: 把小 o 的趋于 0 换成趋于 ∞

枚举

TODO: 找几个例子
后面我们会学习搜索，来实现最一般的枚举

算法可以调用自己。在设计算法时，可以寻找“子问题”，然后调用递归

TODO：找几个例子

抽象贯穿计算机科学的始终。我们后面还会不断地见到子问题的思想。

- 二分
- 排序
- 贪心

TODO: 举几个例子

在课程开始前，我们希望大家具有基本的 C++ 和算法知识，可以熟练切掉橙题。
在上半段课程中，我们会学习各种 CSP-S 级别的算法，期望帮助大家完成普转提的过渡
在下半段课程中，我们会更深入地学习这些算法相关的内容，期望大家在这部分内容上达到 NOIP/省选/NOI 的水平（注意：这三个东西的难度其实没有显著区别）。不过由于这部分内容密度较大，学习起来可能较为困难。

课程节奏

每周一次课 4h + 每周题单 + 一次模拟赛
课下希望大家多多练习，大概每天做两道题左右即可

- 对 OI 概况、利弊的了解
- OI 的一些学习方法
- 一些语法和算法基础的回顾

最后

希望大家学得开心

希望大家学得开心

谢谢大家！